Amlogic

NNIDE Tool Introduction

Revision: 1.0.0

Release Date: 2024-07-01

Copyright

Trademarks

 , and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision History

Issue 1.0.0 (2024-07-01)

This is the Initial release.

Contents

1. Introduction to NNIDE

NNIDE (Neural Network Integrated Development Environment) is a comprehensive neural network development tool developed by Amlogic, based on the nnEngine. It aims to provide a complete platform to support the deployment of deep learning models on edge computing devices. By implementing key functions such as inference, visualization, accuracy analysis, and performance evaluation, NNIDE offers Amlogic's customers an efficient and convenient workflow. This enables customers to quickly assess the feasibility and performance of their models on the Amlogic NN platform, thereby reducing the overall development cycle.

1.1 Introduction to NNIDE_tool directory

NNIDE_TOOL path integrated in the tool:

adla-toolkit-binary-x.x.x.x/bin/adla_plug_in/tools/ NNIDE_tool

Catalog introduction:

```
1、The directory structure is as follows
├── demo
│   ├── amlnn                              #Compiled file of nnide_tool
│   ├── dataset                            #Dataset directory
│   │   ├── aml_dataset_crop_inception         #Classification model test data set(490 pictures)
│   │   └── input                              #Single frame test data
│   ├── demo_adla_compare_buf.py           #Comparison script between the accuracy of the model file saved on the PC and the accuracy of the board end
│   ├── demo_adla_jupyter.ipynb
│   ├── demo_adla_run_datasets.py          #Classification model top1/top5 accuracy test script
│   ├── demo_adla_run_multi_io.py          #Multi-input model inference script
│   ├── demo_adla_run_single_io.py         #Single input model inference script
│   ├── model_file                         #Test related model files
│   │   ├── adla                               #Converted adla file
│   │   └── original                           #Original model files, and conversion scripts
│   ├── NNIDE_package                      #Board end IDE executable file
│   │   ├── android_32                         #Android 32-bit system IDE execution file
│   │   │   └── NNIDE
│   │   ├── android_64                         #Android 64-bit system IDE execution file
│   │   │   └── NNIDE
│   │   ├── linux_32                           #Linux 64-bit system IDE execution file
│   │   │   └── NNIDE
│   │   └── yocto_64                           #Yocto 64-bit system IDE execution file
│   │   └── NNIDE
│   └── requirements.txt                   #PC environment dependency list
└── Readme.txt

Note:
    1. Unzip command: tar -zxf demo.tar.gz
```

2. NNIDE functionality

2.1 API Interface Introduction

2.1.1 amlnn = AMLNN()

API	amlnn = engine.AMLNN(device_type='android', model_type='adla', model_path='./model_file/resnet18-v1-7_int8_A3_A311D2.adla', model_input_size='1 224 224 3', model_input_format='int8', output_type='fp32', run_cycles=1, need_hardware_info=True, need_runtime=True, need_bandwidth=True, need_visualize=True, loglevel='INFO')
function	Initialize the AMLNN class
parameter	device_type：Device_type (android，linux)
	model_type：Model type, default to adla
	model_path：The local path of the adla model
	model_input_size：The shape of the input layer of the model is arranged in n h w c. If it is a multi input model, use # segmentation
	model_input_format：The data type of the model input layer (uint8/int8/int16/flat32/int32/int64), if it is a multi input model, use # segmentation
	output_type：The desired output data type (raw, fp32) is obtained. If raw is selected, the native output buf of the model will be obtained. If fp32 is selected, internal quantization will be performed, and the output data type obtained will be float32
	run_cycles：Inference frequency, can be selected according to needs, default to 1
	need_hardware_info：Is it necessary to obtain hardware information (platform name)
	need_runtime：Is it necessary to obtain the inference time
	need_bandwidth：Is it necessary to obtain bandwidth information
	loglevel：log level (DEBUG，INFO，WARNING，ERROR)
Return value	No

2.1.2 amlnn.init()

API	amlnn.init()
function	Model initialization
parameter	No
Return value	No

2.1.3 amlnn.set_input()

API	amlnn.init_runtime(input_data)
function	Set Model Input
parameter	input_data：After preprocessing, if there are multiple inputs, then you need to reshape their dimensions into one dimension first, and splice them in sequence
Return value	No

2.1.4 amlnn.inference()

API	output_data=amlnn.inference()
function	Model inference and obtaining inference results
parameter	No

Return value	output_data：Model output results

2.1.5 amlnn.destroy()

API	amlnn.destroy()
function	Unload the model and release the related resources
parameter	No
Return value	No

2.1.6 amlnn.similarity_and_distance()

API	amlnn.similarity_and_distance(buf0, buf1)
function	Compare the similarity between two groups of bufs
parameter	buf0, buf1: The buf in the parameter is in the form of an array. You can use len(buf) to check how many bufs there are, and use buf[i].shape to check the dimension of the i-th sub-buf.
Return value	similarity, distance

2.2 Model inference

Through a series of interfaces provided by NNIDE, such as set_input and inference, customers can directly use Python scripts on the PC side to perform specific operations. This completes the inference tasks of quantized models on the board side and obtains the inference results from the edge side. Doing so avoids the direct use of NNSDK interfaces during the testing phase, simplifies the complexity of edge-side inference, and provides customers with a flexible and efficient workflow.

After obtaining the inference results, customers can also add specialized post-processing code in the same Python script to further analyze and process the inference results, thereby implementing the specific application functions of the model.

```
... Model initialization ...
preprocess input_data
amlnn.set_input(input_data)
output_data = amlnn.inference()
postprocess output_data
... Model release ...
```

2.3 Accuracy comparison

Using the similarity_and_distance interface provided by NNIDE, customers can quickly compare the cosine similarity and Euclidean distance between the model saved during the model conversion process and the model board-end to assess the accuracy of the quantized model.

```
... Model inference ...
output_data = amlnn.inference()
output_data_original = customer_get_original_model_output
similarity, distance = amlnn.similarity_and_distance(output_data, output_data_original)
print(f"similarity: {similarity}")
print(f"distance: {distance}")
```

2.4 Performance

By setting the need_runtime and need_bandwidth parameters to True when initializing the AMLNN class and adjusting the loglevel to the INFO level or higher, the user can obtain important performance metrics about the model's operation, including the total elapsed time and bandwidth usage of the NPU, after the model inference is complete. This information will be printed out directly through the command line interface (cmd), providing real-time feedback to the user.

```
amlnn = engine.AMLNN(..., need_runtime=True, need_bandwidth=True, loglevel='INFO', ...)
... Model inference ...
```

3. Introduction to Demo Running Method

Currently, NNIDE supports running on Windows 10 or Ubuntu 20.04 operating system and Python 3.8 environment.

3.1 PC Environment Configuration

3.1.1 adb environment

The PC and development board are connected through the USB interface, and ensure that ADB can properly identify the specified device.

Execute the command in the terminal: adb devices -l, and the target device can be displayed normally.

Execute the command: adb shell to ensure that adb functions normally

3.1.2 python and third-party library environment

After configuring the python3.8 environment, install the dependent libraries

```
pip install -r requirements.txt
```

3.2 Model conversion

The original model and related script files corresponding to the demo are placed in the demo/model_file/original path of the release package.

3.2.1 Script introduction

ide_model_convert.sh is a model conversion script. The --model option is used to specify the original model path. Subsequent customers need to replace it with the path of their own model when converting their own models.

check_input_param.sh is the model information confirmation script, ${adlaininfo_release} -io is followed by the adla model path generated after the model conversion. Similarly, customers need to replace it with their own adla model path.

3.2.2 Convert and confirm model information

The path of the current example demo model has been specified. Customers only need to copy the entire original folder to the toolkit's adla-toolkit-binary-x.x.x.x/demo, then enter the original directory on the command line and execute the command ./ide_model_convert.sh. The model conversion process can be completed, and the generated adla file is in the output folder of the current path.

The path of the current example demo model has been specified. After completing the model conversion, directly execute ./check_input_param.sh on the command line to see the relevant information of the model input and output.

These generated adla models can then be copied to the demo/model_file/adla path of the release package (a set of converted adla models with platform suffixes added to them are currently provided in this path)

3.3 Start the NNIDE service

NNIDE files for different environments are placed under the demo/NNIDE_package path of the release package. Customers can select the corresponding NNIDE files according to the environment corresponding to the platform used.

3.3.1 Android

Open the terminal, enter the demo directory of the release package, and execute the following commands in sequence

```
adb root
adb remount
adb push your_path/NNIDE /vendor/bin     (your_path is the path of NNIDE selected by the customer)
adb shell
cd /vendor/bin
chmod +x NNIDE
./NNIDE
```

If you see the following print, it means the NNIDE service started successfully

```
nnEngine start to work, listening on port 8308
nnEngine start to work, listening on port 8309
nnEngine start to work, listening on port 8310
```

3.3.2 Linux

Open the terminal, enter the demo directory of the release package, and execute the following commands in sequence

```
adb push your_path/NNIDE /usr/bin     (your_path is the path of NNIDE selected by the customer)
adb shell
cd /usr/bin
chmod +x NNIDE
./NNIDE
```

If you see the following print, it means the NNIDE service started successfully

```
nnEngine start to work, listening on port 8308
nnEngine start to work, listening on port 8309
nnEngine start to work, listening on port 8310
```


3.4 Running the demo

Currently, 5 sample demos are provided, namely

single input single output model	demo_adla_run_single_io.py
Multiple input multiple output model	demo_adla_run_multi_io.py
Loop through the data set	demo_adla_run_datasets.py
Compare output buf differences	demo_adla_compare_buf.py
jupyter environment demo	demo_adla_jupyter.ipynb

For single-input single-output models, multiple-input multiple-output models, run the demo of the data set in a loop. After completing 3.1, 3.2, and 3.3, open another terminal and directly execute the command python xxx.py on the command line to run it.

For the demo comparing the output buf difference between the original model and the quantified model, since the original model is tflite, you need to configure an environment that can run tflite. Just pip install tflite_runtime, and then execute the command python xxx.py on the command line to run it.

For interactive demos running in the jupyter environment, you need to configure the jupyter environment first, just pip install jupyter, and then execute jupyter notebook on the command line to enter the web version of jupyter, open demo_adla_jupyter.ipynb on the web page, and click on the upper left corner Triangle to step through blocks of code

4. FAQ

4.1 Debugging means

Step 1. On the PC side, set the log level to DEBUG level, loglevel='ERROR'

Step 2. On the board side, set the log level to DEBUG level, export NN_ENGINE_LOG_LEVEL=4

Step 3. Execute the Python script again

4.2 Input and Output

Input: The input data input_data given to the amlnn.set_input() interface is pre-processed quantified data, and the format needs to be nhwc. If there are multiple inputs, their dimensions need to be reshaped into one dimension first, and then Spliced in sequence.

Output: The output_data obtained from the amlnn.inference() interface is the data after npu inference. It is an array. You can use len(output_data) to view how many output bufs there are, and use output_data[i].shape to view each. Output the shape of buf. The buf format is nhwc. Customers can add post-processing code based on the python side to verify whether the model accuracy is normal.